Engineering | MBA | MCA

Marwadi Education Foundation

Marwadi Education Foundation's Group of Institutions

Faculty of Computer Applications

MCA Sem- III

Fundamental of Java Programming

(630002)

Unit – 5
Files, Thread and Multithreading

Introduction

Streams are the most important part in creating the applications like Data Input from keyboard and output on the System Output Devices. Actually, uptil now we have used input and output streams in our applications but we don't know explicitly that we are using streams.

In java, for the Input / Output (IO) operations, we have two major packages. One is the java.io package which was available from java 1.0 and then later from java 1.4 we get java.nio package.

Introduction

Remember that : the java.nio package is not a replacement of the java.io package.

Both are different ways of performing IO operations. The java.io package has classes and interfaces which help in file management and stream-based IO, and the java.nio has classes and interfaces for the buffer-based IO. Various important classes of java.io package are given in next table.

Classes of java.io package

Class	Usage
File	Used for file management
InputStream	The top level binary stream class for getting input
OutputStream	The top level binary stream class for giving output
Reader	The top level stream class for reading text
Writer	The top level stream class for writing text output
RandomAccessFile	Used for reading and writing to a file
DataInput	An interface to deal with input related to binary types
DataOutput	An interface to deal with output related to binary types
ObjectInput	An interface to deal with input related to binary types including reference type
ObjectOutput	An interface to deal with output related to binary types including reference type
Serializable	A marker interface to enable serialization of instances of a certain class.

Files and Directories

The file class gives us the information regarding the properties of a file or directory (Folder). These include its read and write permissions, time of last modification, and length. It is also possible to determine the files that are contained in a directory. This is valuable because you can build an application that navigates a directory hierarchy. New directories can be created, and existing files and directories may be deleted or renamed.

Files and Directories

The File class provides following three constructors.

(1) File (String path)

(2) File (String directorypath, String path)

(3) File (File Directory, String filename)

The first form has one parameter that is the path of the file or directory. The second form has two parameters. These are the path to a directory and the name of a file in that directory. The last form also has two parameters. These are File object for a directory and name of the file.

Files and Directories

Remember that : All of these constructors throw a NullPointerException if path or filename is null.

This class also defines two character constants. These are **separatorChar** and **pathSeparatorChar.** The first is the character that separates the directory and file portions of a file name. The second is the character that separates components in a “path-list”. Obviously both of these are platform dependent.

Files and Directories

Various methods of the class are as follows :

Method	Usage
boolean canRead()	Returns true if the file is in readable mode otherwise returns false
boolean canWrite()	Returns true if the file is in write mode otherwise returns false
boolean delete()	Delete the given file. Returns true if the file successfully deleted otherwise returns false
boolean equals(Object obj)	Returns true if the current object and obj refer to the same file otherwise returns false
boolean exist()	Returns true if the file exist otherwise returns false
String getAbsolutePath()	Returns the absolute path of the file
String getCanonicalPath()	Returns the canonical path of the file
String getName()	Returns the name of the file.

Files and Directories

Various methods of the class are as follows :

Method	Usage
String getParent()	Returns the name of parent of the file
String getPath()	Returns the path of the file
boolean isAbsolute()	Returns true if the file path name is absolute otherwise returns false
boolean isDirectory()	Returns true if the file is a directory otherwise returns false
boolean isFile()	Returns true if the file is not a directory otherwise returns false
boolean isHidden()	Returns true if the file is hidden file otherwise returns false
long lastModified()	Returns the no. of milliseconds between epoch and the time of last modification of this file
long length()	Returns the no. of bytes in the file

Files and Directories

Various methods of the class are as follows :

Method	Usage
String[] list()	Returns an array of string of all file names located in the current directory
boolean mkdir()	Creates a directory with the name of this file. Returns true if the directory created otherwise returns false
boolean mkdirs()	Creates a directory with the name of this file. Any missing parent directories are also created. Returns true if the directory created otherwise returns false
boolean renameTo(File newname)	Renames the file or directory to newname. Returns true if successful otherwise returns false.

ex\ex81.java

Character Streams

A stream is an abstraction for a source or destination of data. It enables you to use the same techniques to interface with different types of physical devices. E.g. an input stream may read its data from a keyboard, file or memory buffer. An output stream may write its data to a monitor, file or memory buffer.

In java we can also use other types of devices also like to get input from the lift button or temperature.

Character Streams

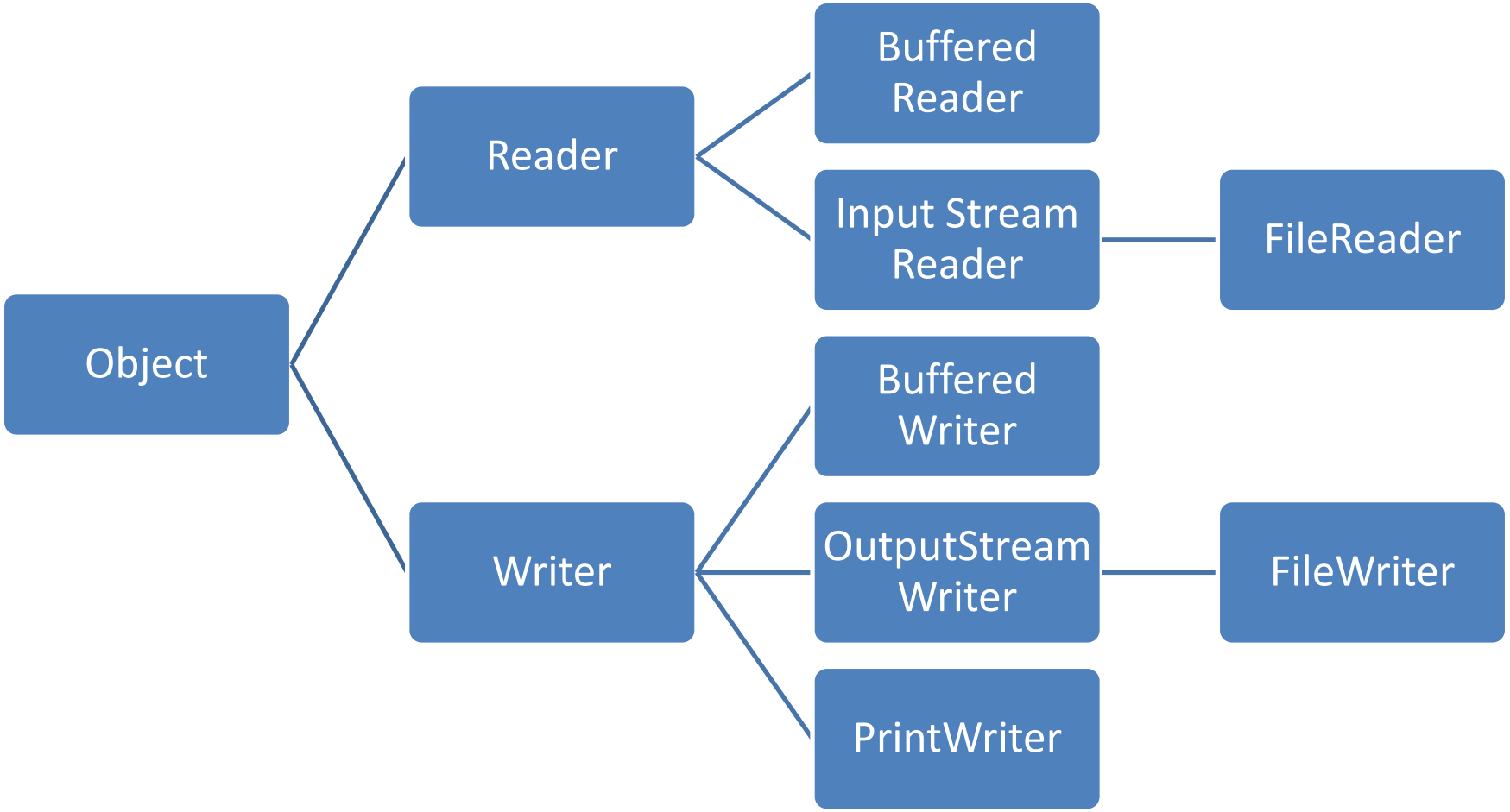
There are two types of streams : byte and character. Byte streams allow you to read and write binary data. E.g. an application that simulates the behavior of an electric circuit can write a sequence that simulates the behavior of an electric circuit can write a sequence of float values to a file. These would represent the value of a signal over a time interval. This binary data could later be retrieved for analysis.

Character Streams

Character streams allow you to read and write characters and strings. An input character stream converts bytes to characters. An output character stream converts characters to bytes.

Streams can be understand by the following figure.

Streams


Object
Reader
Writer
Buffered Reader
Input Stream Reader
FileReader
Buffered Writer
OutputStream Writer
FileWriter
PrintWriter

The Writer Class

The abstract Writer class defines the functionality that is available for all character output streams. It has following constructors.

(1) Writer()

(2) Writer(Object obj)

The first form synchronizes on the Writer object and the second form synchronizes on object.

The Writer Class

Various methods of the class are as follows :

Method	Usage
void close()	Closes the output stream. Note : Must be implemented by a subclass
void flush()	Writes any buffered data to the physical device represented by the stream. Note : Must be implemented by a subclass
void write(String s)	Writes s to the stream
void write(String s, int start, int length)	Writes no. of characters from given start and for length no. of characters

The OutputStreamWriter Class

The OutputStreamWriter class extends writer class. It converts a stream of characters to a stream of bytes. This is done according to the rules of a specific character encoding. It provides two constructors.

(1) OutputStreamWriter(OutputStream os)

(2) OutputStreamWriter(OutputStream os, String encoding)

Here os is the output stream and encoding is the name of a character encoding. The first form of the constructor uses the default character encoding of the user's machine.

The OutputStreamWriter Class

The getEncoding() method returns the name of the character encoding. It has following syntax :

String getEncoding()

This method will returns the encoding method of the string encoding.

All the methods of writer class will be used as it is in this class.

The FileWriter Class

The FileWriter class extends OutputStreamWriter and outputs characters to a file. This class has three constructors.

(1) FileWriter(String filepath) throws IOException

(2) FileWriter(String filepath, boolean append) throws IOException

(3) FileWriter(File fileObj) throws IOException

here the filepath is the full path name of a file and fileObj is a File object that describes the file. If append is true, characters are appended to the end of the file otherwise the existing content of the file are overwritten.

The Reader Class

The abstract Reader class defines the functionality that is available for all character input streams. This class provides only one constructor which initialize the class.

Various methods of the class are as follows.

The Reader Class

Various methods of the class are as follows :

Method	Usage
void close()	Closes the input stream. Remember that further read attempts generate an IOException. Note : must be implemented by a subclass.
void mark()	Places a mark at the current point in the input stream that will remain valid until number of characters are read.
boolean markSupported()	Returns true if mark() / reset() methods are supported on the stream or not
int read()	Reads a single character from the stream.
boolean ready()	Returns true if the next read() will not wait
void reset()	Reset the input pointer to the previously set mark.
int skip(long numChars()	Skips the no. of characters in reading stream

The InputStreamReader Class

The InputStreamReader class extends Reader. It converts a stream of bytes to a stream of characters. This is done according to the rules of specific character encoding.

This class provides two constructors

(1) InputStreamWriter(InputStream is)

(2) InputStreamWriter(InputStream is, String encoding)

Here is is the input stream and encoding is the name of a character encoding. The first form of the constructor uses the default character encoding of the user's machine.

The InputStreamReader Class

The getEncoding() method returns the name of the character encoding. It has following syntax :

String getEncoding()

This method will returns the encoding method of the string encoding.

All the methods of reader class will be used as it is in this class.

The FileReader Class

The FileReader class extends InputStreamReader and inputs characters from a file. Its constructors are as follows :

(1) FileReader(String filepath)

(2) FileReader(File fileObj)

This will throw a FileNotFoundException. Here the filepath is the full path name of a file and fileObj is a File object that describes the file.

ex\ex82.java

ex\ex83.java

Exercise

(1) Write an application that reads a file and counts the number of occurrences of each digit between 0 to 9. Supply the file name as command line argument.

(2) Write an application that reads a file and counts the no. of occurrences of each vowel i.e. a,e,i,o and u. Supply the file name as command line argument.

(3) Write an application that copies one character file to a second character file. Both file names would be provided by command line arguments.

The Buffered Character Streams

There are mainly two classes for Buffered Character Streams they are namely (1) BufferedWriter and (2) BufferedReader.

The BufferedWriter class extends Writer and it is useful to buffer the output to a character stream. It provides two constructors :

(1) BufferedWriter(Writer w)

(2) BufferedWriter(Writer w, int bufSize)

The first form creates a buffered stream using a buffer with a default size. In the second form the size of the buffer is specified by us.

The Buffered Character Streams

The BufferedReader class extends Reader and buffers the input from a character stream. Its constructors are as follows :

(1) BufferedReader(Reader r)

(2) BufferedReader(Reader r, int bufSize)

The first form creates a buffered stream using a buffer with a default size. In the second, the size of the buffer is specified by the bufSize.

This class has all the methods of Reader. In addition the **readLine()** method reads new line terminated string from a character stream.

The Buffered Character Streams

ex\ex84.java

ex\ex85.java

ex\ex86.java

Exercise

(1) Write an application that reads a file, converts each tab character to a space character and writes its output to another file. Both file names are given as command line arguments.

(2) Write an application that reads and processes string from the console. Reverse the sequence of character in each string and then display it.

The PrintWriter Class

The PrintWriter class extends Writer and displays string equivalents of simple types such as int, float, char and objects. Its functionality is valuable because it provides a common interface by which many different data types can be output.

The constructors are as follows :

(1) PrintWriter(OutputStream os)
(2) PrintWriter(OutputStream os, boolean flush)
(3) PrintWriter(Writer w)
(4) PrintWriter(Writer w, boolean flush)

The PrintWriter Class

Here the flush will control that whether java flushes the output stream every time a newline (\n) character is output. If the value of flush is true then flushing will automatically takes place with each and every new line.

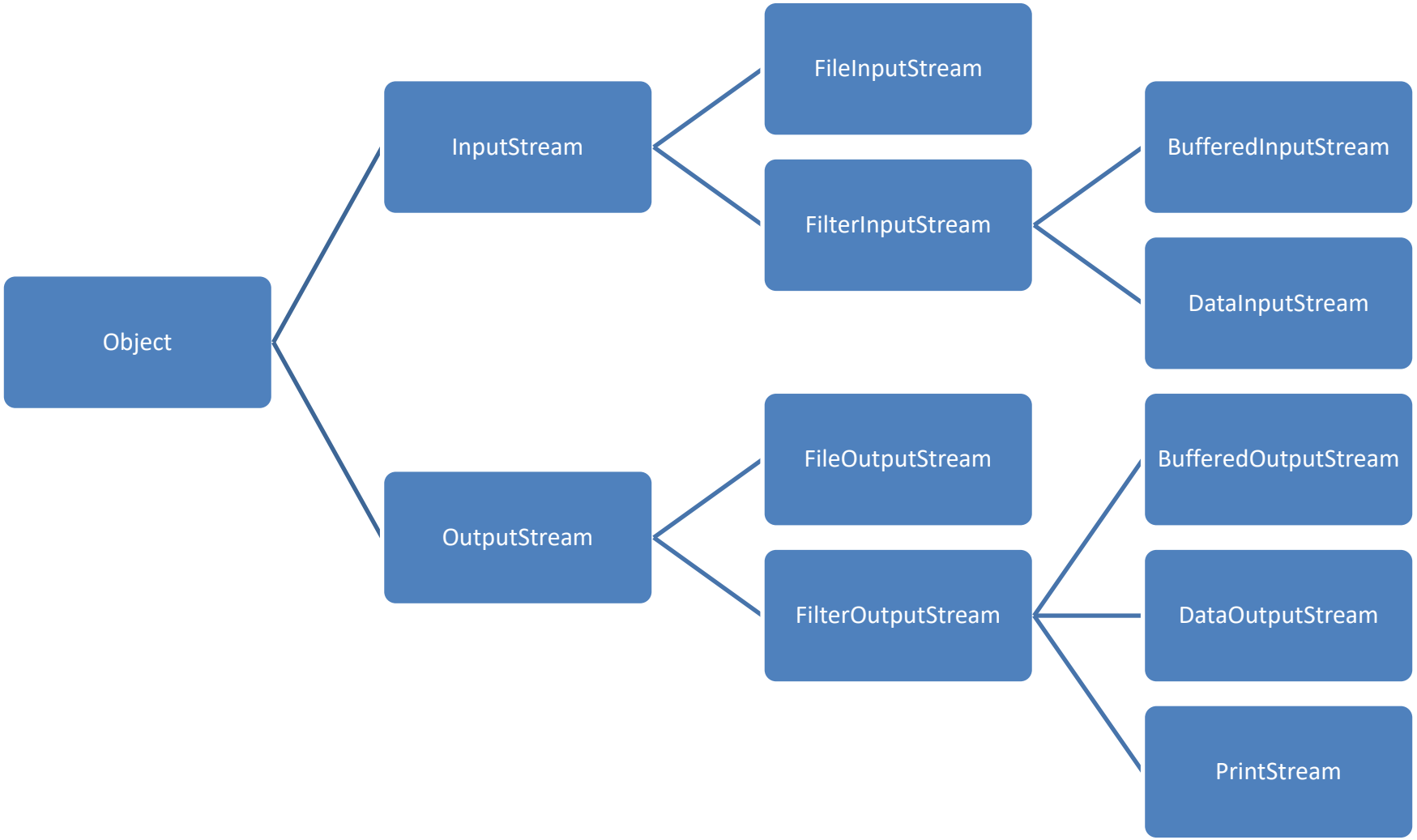
Java's PrintWriter object supports the print() and println() methods for all types including Object. If an argument is not of simple type then it will call the object's toString() method and display the string that is returns from this method.

ex\ex87.java

Byte Streams

Byte Streams allows the programmer to work with the binary data in a file. The next figure will shows us the byte streams provided by the java.io package. These classes are OutputStream, FileOutputStream, FilterOutputStream, BufferedOutputStream, DataOutputStream, PrintStream for output and for input FileInputStream, FilterInputStream, BufferedInputStream and DataInputStream.

Byte Streams



The OutputStream Class

The OutputStream class defines the functionality that is available for all byte output streams. Various methods of the class are as follows :

Method	Usage
void close()	Close the output Stream
void flush()	Flushes the output stream
void write(int i)	Writes the lowest 8 bits of I to the stream
void write(byte buffer[])	Writes buffer array to the stream
void write(byte buffer[], int index, int size)	Writes the buffer array starting at position index for given size

The FileOutputStream Class

The FileOutputStream class extends OutputStream and allows you to write binary data to a file. Its most commonly used constructors are :

(1) FileOutputStream(String filePath)
(2) FileOutputStream(String filepath, boolean app)
(3) FileOutputStream(File fileObj)

Here the filepath is full path name of a file and fileObj is a File object that describes the file. If append is true, characters are appended to the end of file otherwise the existing content of the file will be overwritten.

The FilterOutputStream Class

The FilterOutputStream class extends OutputStream. It is used to filter output and provides this constructor :

FilterOutputStream(OutputStream os)

Here the os is the output stream to be filtered.

Remember that : We are not allowed to directly use the FilterOutputStream. Instead of that you must create a subclass to implement the desired functionality.

The BufferedOutputStream class

The BufferedOutputStream class extends FilterOutputStream and buffers the output to a byte stream. Its constructors are :

(1) BufferedOutputStream(OutputStream os)

(2) BufferedOutputStream(OutputStream os, int bufSize)

The first argument in both constructors is a reference to the output stream. The first form creates a buffered stream by using a buffer with a default size. In second form the buffer is specified by bufSize.

The DataOutputStream Class

The DataOutputStream class extends FilterOutputStream and implements DataOuput. It allows you to write the simple java types to a byte output stream. This class has only one constructor which as follows :

DataOutputStream(OutputStream os)

Here the os is the output Stream.

The DataOutput Interface

The DataOutput interface defines methods that can be used to write the simple java types to a byte output stream. Various methods of this interface are as follows :

Note that : All the methods of DataOutput interface will throw an IOException.

The DataOutput Interface

Various methods of the interface are as follows :

Method	Usage
void write(int i)	Writes i to the stream
void write(byte buffer[])	Writes byte array to the stream
void write(byte buffer[], int index, int size)	Writer size bytes from buffer starting at position index to the stream
void writeBoolean(boolean b)	Writes b to the stream
void writeBytes(String s)	Writes s to the stream
void writeChars(String s)	Writes s to the stream
void writeDouble(double d)	Writes d to the stream
void writeFloat(float f)	Writes f to the stream
void writeInt(int i)	Write i to the stream
void writeLong(long l)	Writes l to the stream
void writeShort(short s)	Writes s to the stream

The PrintStream Class

The PrintStream class extends FilterOutputStream and provides all of the formatting capabilities we have been using from System.out from the beginning of our java discussion.

Remember that : the static System.out variable is a PrintStream.

The PrintStream class has two constructors which are as follows :

(1) PrintStream(OutputStream os)

(2) PrintStream(OutputStream os, boolean flush)

The PrintStream Class

Here the flush will control whether java flushes the output stream every time a new line character is output. If flush is true flushing will take place automatically otherwise not.

Java's PrintStream objects support the print() and println() methods for all types including Object. If an argument is not a simple type then the PrintStream methods will call the object's toString() method and then print the result.

The InputStream Class

The InputStream class defines the funcionlity that is available for all byte output streams. Various methods of the class are as follows :

Method	Usage
int available()	Returns the number of bytes available for reading.
void close()	Closes the input stream
void mark(int num)	Places a mark at current point in the input stream. It remains valid till num bytes are read
boolean markSupported()	Returns true if the marks is supported otherwise returns false
int read()	Reads one byte from the input stream
void reset()	Reset the input pointer to the previous mark
int skip(int num)	Skips no. of bytes of input and returns the no. of bytes skipped.

The FileInputStream class

The FileInputStream class extends InputStream and allows you to read binary data from a file. Its constructors are as follows :

(1) FileInputStream(String filepath)

(2) FileInputStream(File fileObj)

Here filepath is a full path name of a file and fileObj is a File Object.

The FilterInputStream class

The FilterInputStream class extends InputStream and filters an input stream. It has following constructor

FilterInputStream(InputStream is)

here the is is the input stream to be filtered.

Remember that : We are not allowed to directly instantiate FilterInputStream class. Instead of that we must have to create a subclass to implement the desired requirement.

The BufferedInputStream class

The BufferedInputStream class extends FilterInputStream and buffers the input from a byte stream. Two constructors of the class are as follows:

(1) BufferedInputStream(InputStream is)

(2)BufferedInputStream(InputStream is, int bufSize)

The first argument to both constructors is a reference to the input stream. In first constructor the buffer size will be default and in second we have to specify the buffer size.

The DataInputStream class

The DataInputStream class extends FilterInputStream and implements DataInput. It allows you to read the simple Java types from a byte input stream. It has one constructor as follows:

DataInputStream(InputStream is)

here is is the input Stream

The DataInput interface

The DataInput interface defines methods that can be used to read the simple java types from a byte input stream. Various methods of the interface are as follows :

Method	Usage
boolean readBoolean()	Reads and return boolean from the stream
byte readByte()	Reads and returns a byte from the stream
char readChar()	Reads and returns a char from the stream
double readDouble()	Reads and returns a double from the stream
float readFloat()	Reads and returns a float from the stream
int readInt()	Reads and returns an int from the stream
long readLong()	Reads and returns a long from the stream
int skipBytes(int n)	Skips the n number of bytes

FileOutputStream Example : ex\ex88.java
FileInputStream Example : ex\ex89.java
BufferedOutputStream Example : ex\ex90.java
BufferedInputStream Example : ex\ex91.java
DataOutputStream Example : ex\ex92.java
DataInputStream Example : ex\ex93.java

Exercise :

(1) Write one application that splits a large file into smaller files and a second application that merges these smaller files to re-create the large file. This is useful when we want to mail large file.

(2) Write one application that writes the first 15 numbers of the Fibonacci series to a file. Also develop a one more application which reads these 15 nos. from the file. For both application specify the name of the file as command-line argument.

Random Access Files

When we use stream classes we have to remember that that allows us to use through sequential access for reading and writing the data in a file. The **RandomAccessFile** class allows you to write programs that can seek to any location in a file and read or write data at that point.

This type of functionality is very valuable in some programs like when we want to manage a set of data records that are stored in a file.

This class implements DataInput and DataOutput interfaces.

Random Access Files

Various methods of the class are as follows :

Method	Usage
void close()	Close the file
long getFilePointer()	Returns the current position of the file pointer. This identifies the point at which the next byte is read or written
long length()	Returns the number of bytes in the file.
int read()	Reads and returns a byte from the file.
int read(byte buffer[], int index, int size)	Attempts to read size bytes from the file and places these in buffer starting at position index. Returns the number of bytes actually read
void seek(long n)	Positions the file pointer at n bytes from the beginning of the file. The next read and write will occur from that position
int skipBytes(int n)	Adds n to the file pointer. Returns the actual number of bytes skipped. If n is negative then no bytes will be skipped.

Random Access Files

ex\ex94.java

Exercise

(1) Write a program to display the content of file in reverse order sequence. Provide the file name as command line argument

Multithreading

Introduction

In modern Operating Systems like Windows 95 / 98 or XP, we know that they can execute several programs simultaneously. This is known as **multitasking.** In system's terminology it is called as **multithreading.**

In other words MultiThreading is a concept where the program (process) is divided into two or more subprograms (processes), which can be implemented at the same time in parallel.

Introduction

For example : One subprogram is displaying an animation on the screen while another subprogram will build the next animation to be displayed. This is something like dividing a task into subtasks and assigning them to different people for execution independently and simultaneously.

In most of our computers we have single processor and therefore in reality the processor is doing one thing at a time. But the processor switches between the processes so fast that it appears that they are being done simultaneously.

Introduction

A thread is similar to a program that has a single flow of control. It has beginning, a body and an end. So in other words we can say that all our programs till to date are single-threaded programs.

A unique feature of java is its support for multithreading. A program that contains multiple flows of control is known as multithreaded program.

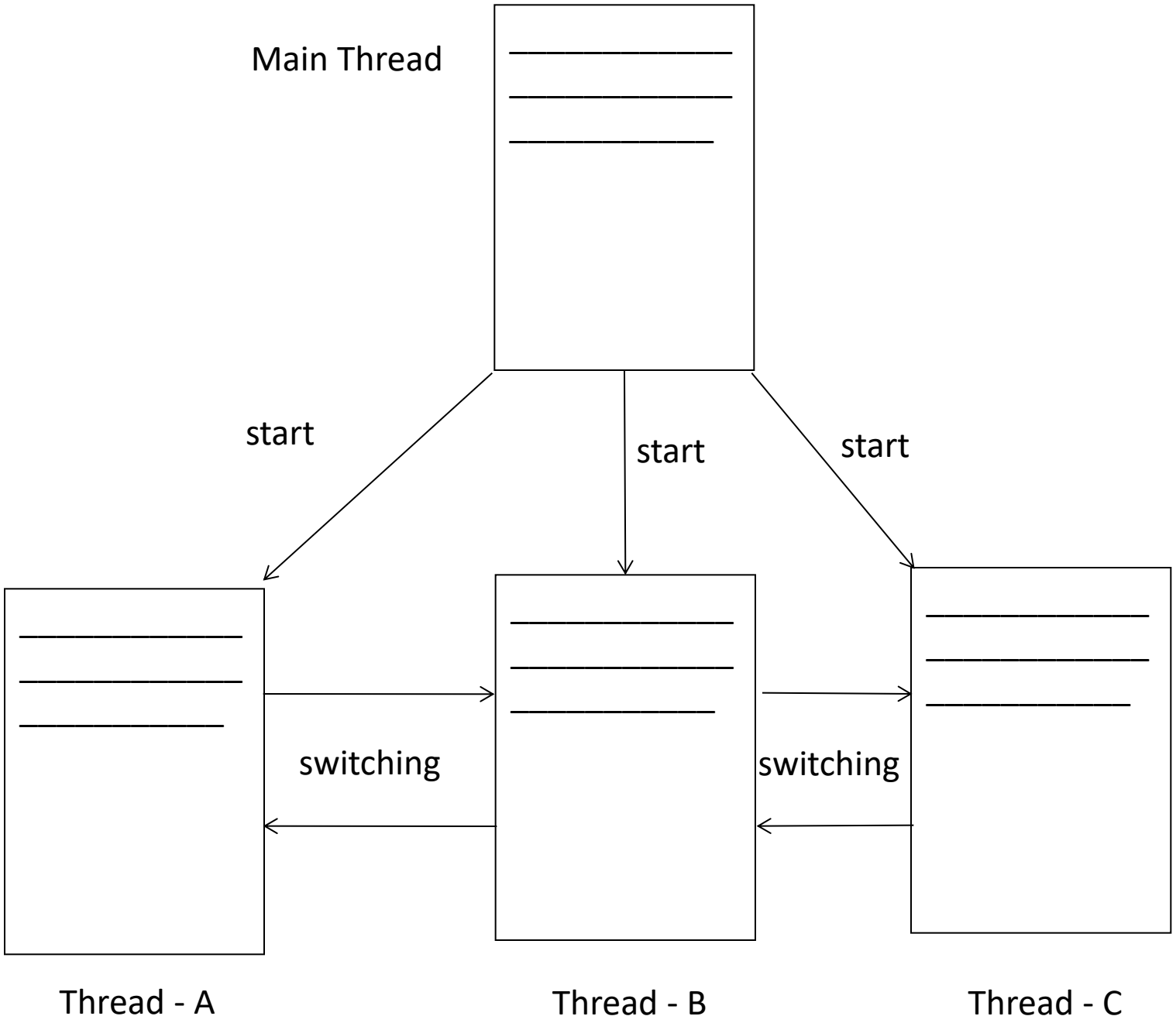
Remember that : The Java Virtual Machine (JVM) manages these and schedules them for execution.

Introduction

A thread is a sequence of execution within a process and we may think thread as a “lightweight” process.

There may be several threads in one process and Java Virtual Machine (JVM) will manage the schedules of execution of them.

The time needed to perform context switching from one thread to another is quite less than that required for performing such a change between processes - it is a benefit of Threding.


Main Thread
start
start
start
switching
switching
Thread - A
Thread - B
Thread - C

Creation of New Thread

Java provides two ways to create a new thread.

(1) By Extending the Thread class which is available in package java.lang.Thread

e.g.

```
class myThread extends Thread
{
                    ……..
                    ……..
}
```

ex\ex23.java

Creation of New Thread

Java provides two ways to create a new thread.

(2) Implement the Runnable interface from the package java.lang.Runnable

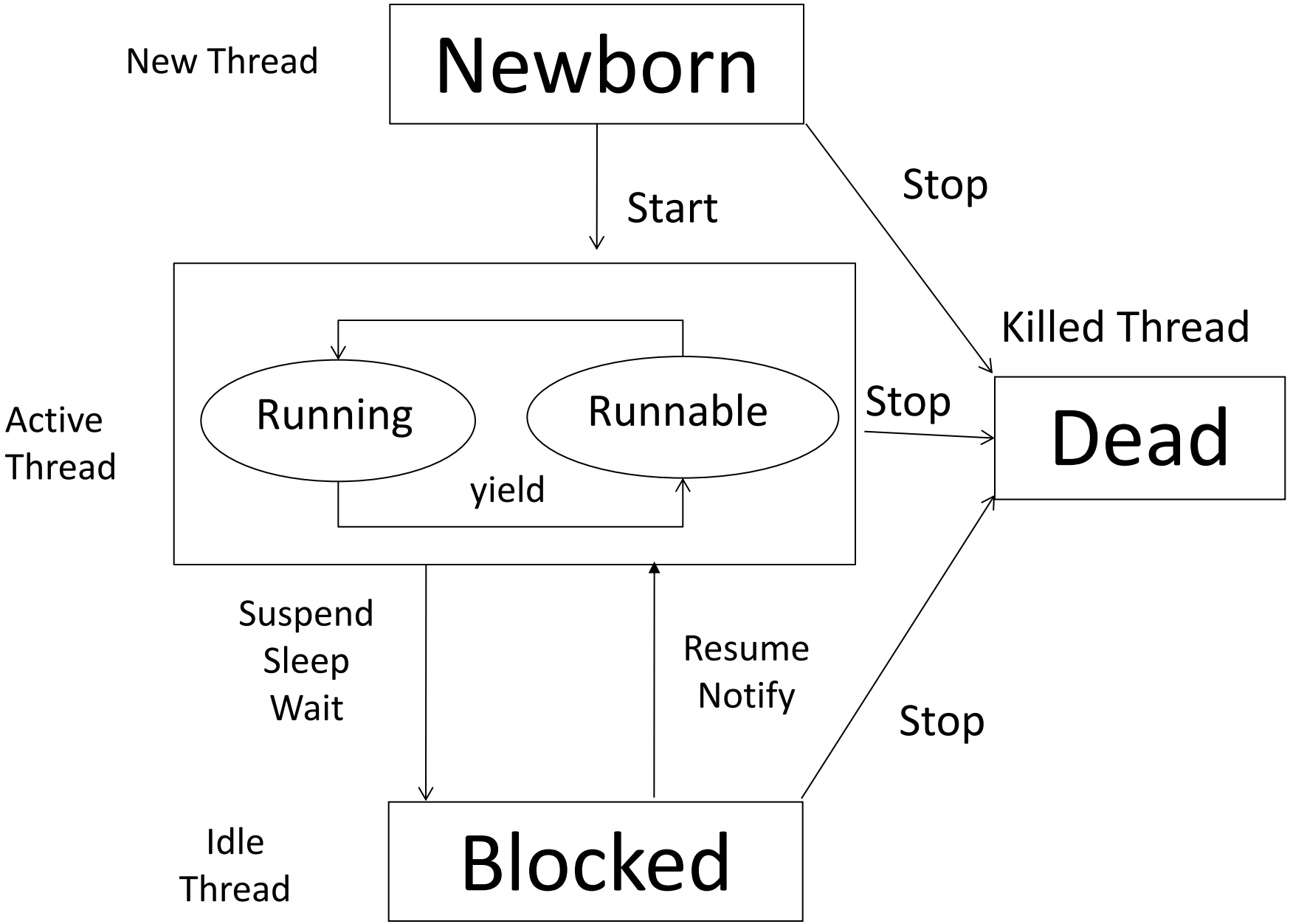
e.g.

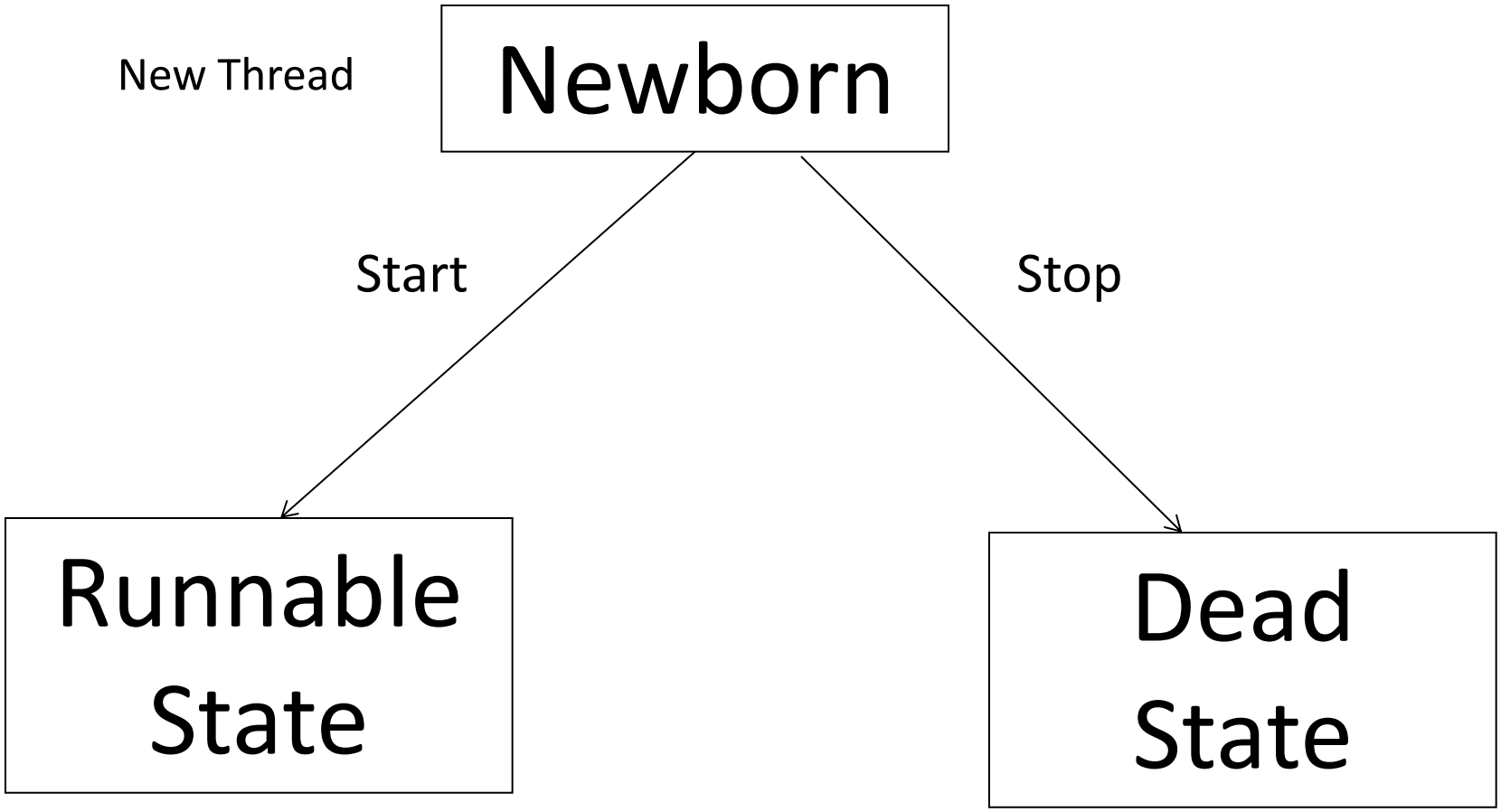
```
class myThread implements Runnable
{
        ……..
        ……..
}
```

Life Cycle of a Thread

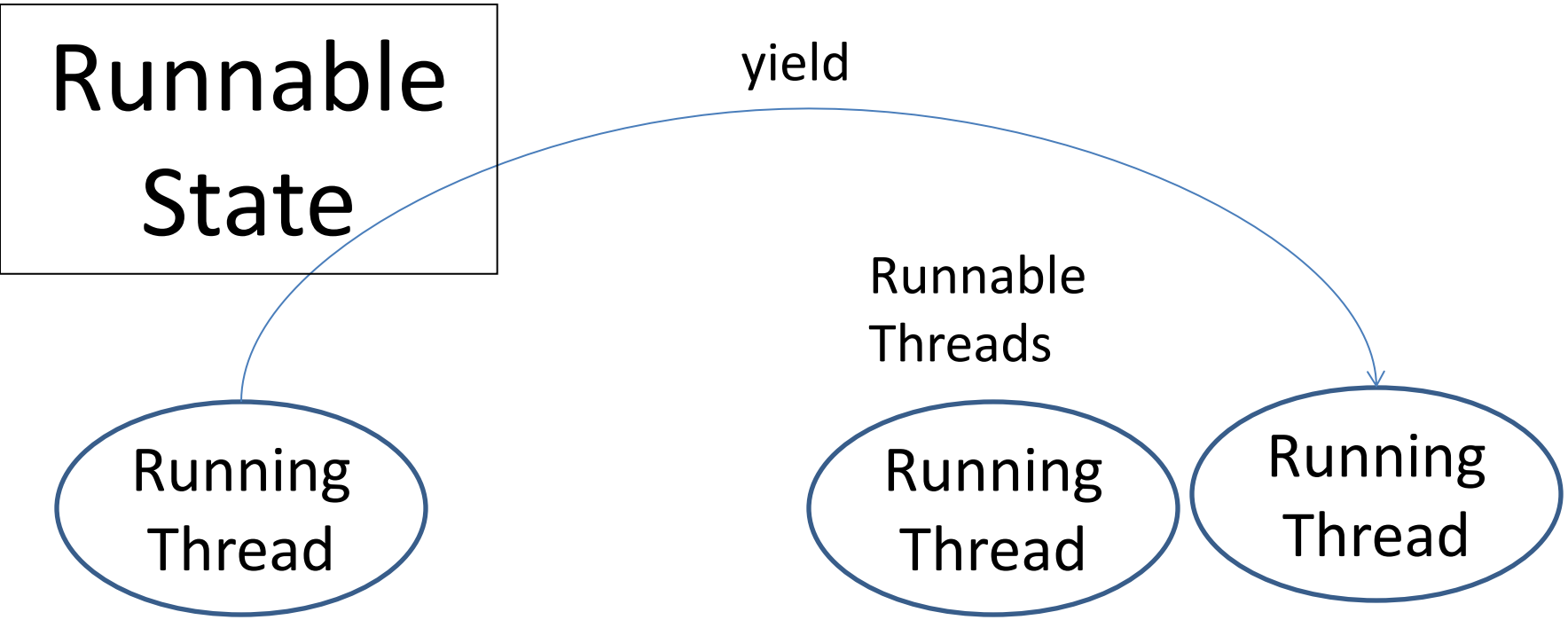
When we are using a thread in our program. It always have a specific life cycle for all threads. In a life cycle of a thread there are 5 stages like

(1) Newborn state
(2) Runnable state
(3) Running state
(4) Blocked state
(5) Dead state


New Thread
Newborn
Start
Stop
Killed Thread
Active
Thread
Running
Runnable
yield
Stop
Dead
Suspend
Sleep
Wait
Resume
Notify
Stop
Idle
Thread
Blocked

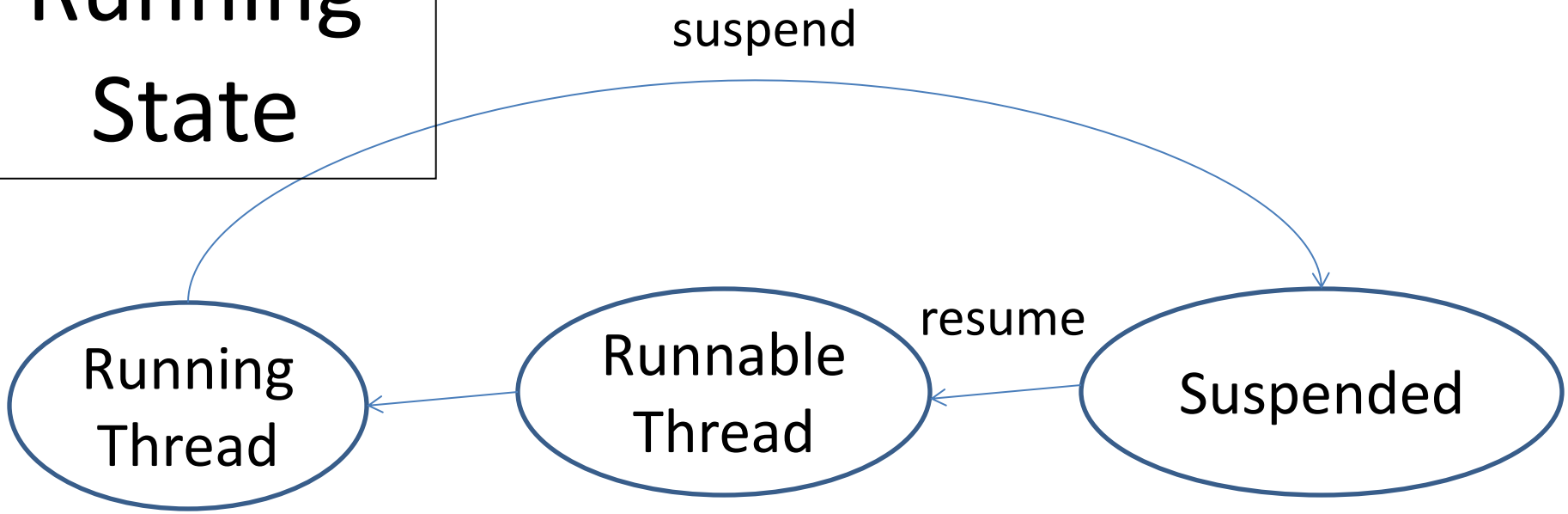


When we create any thread object, it is said to be in newborn state. At this stage we can do two things (1) start (2) stop.



The Runnable state means that the thread is ready for execution and is waiting for availability of the processor. The thread is in queue now and waiting for execution. Generally they will be executed in FCFS manner.

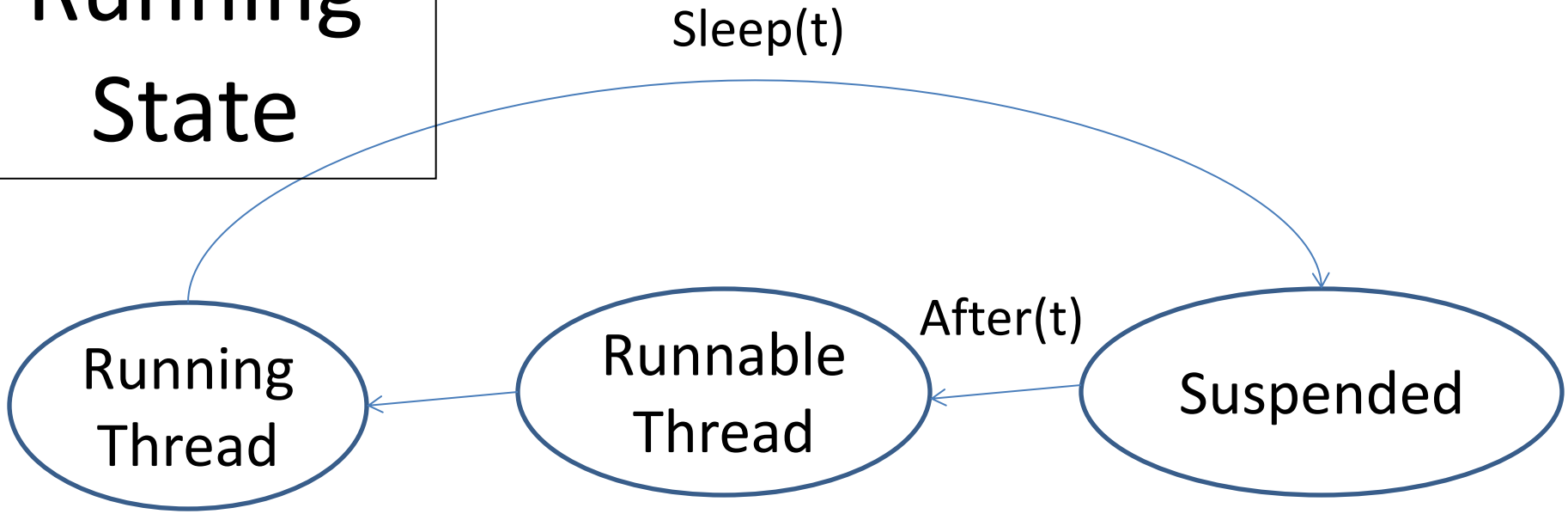
Running State



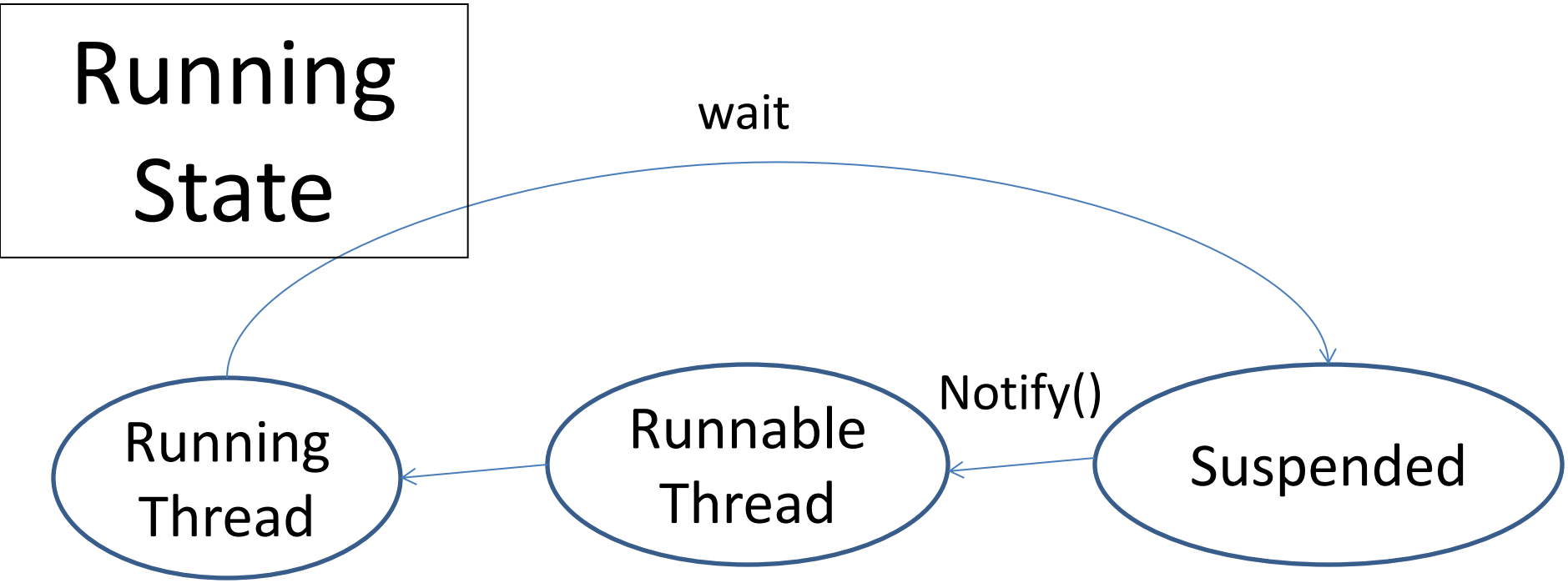
In Running state means the processor has given its time to thread for its execution. The thread runs till it gives control to another.

There is one method called suspend() which can be again put in running by resume() method.

Running State



We can make a thread to sleep. We can put a thread to sleep by using sleep(time) where time is in miliseconds. The thread will re-enter into the runnable state as soon as the time is over.



It has been told to wait until some event occurs. This is done using wait() method. The thread can be scheduled to run again using notify() method.

Various Methods of Thread Class

Method	Description
void destroy()	To destroy the Thread from the memory
String getName()	Returns the name of the thread.
int getPriority()	Returns the priority of the thread.
void interrupt()	Interrupts the current thread.
boolean interrupted()	Tests whether the thread is interrupted or not
boolean isAlive()	Tests whether the thread is alive or not
void join()	Wait for the calling thread to die.
void join(long millis)	Wait atmost <millis> millisecond for this thread to die.

Various Methods of Thread Class

Method	Description
void resume()	Resumes the thread execution
void setPriority(int newPriority)	Change the priority of the thread
void sleep(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
void start()	Causes this thread to begin execution; the Java Virtual Machine calls the run method of this thread.
void stop()	Stops the execution of current thread
void suspend()	Suspends the current thread execution until the resume method has been called.
void yield()	Causes the currently executing thread object to temporarily pause and allow other threads to execute.

ex\ex24.java

Thread Priority

In Java each thread will be assigned a priority, which affects the order of its execution.

If the two or more threads have same priority then they have been given equal treatment by Java scheduler and they will be processed on FCFS basis.

On the other hand we – the programmers can also set the priority by using setPriority() method.

Thread Priority

In setPriority() method we have to pass integer no. or constant according to our requirement just like

threadname.setPriority(intnumber)

Thread class has sevelral constants also they are

MIN_PRIORITY = 1

NORM_PRIORITY = 5

MAX_PRIORITY = 10

Thread Priority

Remember that the default setting is NORM_PRIORITY

When java program gets a new thread with higher priority then presently running thread then the presently running thread will move into the runnable state and new thread will come in to execution.

ex\ex25.java

Runnable Interface

From the first slide we know that threads can be created in two ways :

(1) By extending Thread class

(2) By implementing Runnable Interface.

Runnable Interface will work same as extending thread class. But it is useful when we need to extend some other class at that time this implementation will be much useful.

ex\ex26.java

Synchronization

Synchronization is a problem that can occur when multiple threads access shared data and we have to take care of it.

If the multithreading does not handle properly then data corruption can also be occurred which will leads us to system failure also.

******** **Bank Example** **********

Solution to Synchronization

One of the solution is use of synchronized keyword. When we use a synchronized against method, it automatically acquires a lock on the object. The lock is automatically relinquished when the method completes. So only one thread has a lock at any time so only one thread will be executed in synchronization situation.

ex\ex27.java

DeadLock

Deadlock is an error that can be encountered in multithreaded programs. It occurs when two or more threads wait indefinitely for each other to relinquish locks Assume that thread-1 has a lock on object-1 and waits for a lock on object-2. Thread-2 holds a lock on object 2 and waits for a lock on object-1. Neither if these threads may proceeds because each waits forever for the other to relinquish the lock it needs.

DeadLock

Deadlock situation can also arise when it involves more than two threads also. Assume that thread-1 wait for lock held by thread-2. Thread-2 waits for a lock held by Thread-3 and Thread-3 waits for a lock held by Thread-1.

ex\ex28.java
ex\ex29.java

Grouping Threads

Every Java thread is a member of a thread group. Thread groups provide a mechanism for collecting multiple threads into a single object and manipulating those threads all at once, rather than individually. For example, you can start or suspend all the threads within a group with a single method call. Java thread groups are implemented by the ThreadGroup class in the java.lang package.

Grouping Threads

The runtime system puts a thread into a thread group during thread construction. When you create a thread, you can either allow the runtime system to put the new thread in some reasonable default group or you can explicitly set the new thread's group. The thread is a permanent member of whatever thread group it joins upon its creation. **Remember that : you cannot move a thread to a new group after the thread has been created.**

Need for Grouped Threads

Thread groups offer a convenient way to manage groups of threads as a unit. This is particularly valuable in situations in which you want to suspend and resume a number of related threads. For example, imagine a program in which one set of threads is used for printing a document, another set is used to display the document on the screen, and another set saves the document to a disk file. If printing is aborted, you will want an easy way to stop all threads related to printing. Thread groups offer this convenience.

The Default Thread Group

If you create a new Thread without specifying its group in the constructor, the runtime system automatically places the new thread in the same group as the thread that created it (known as the current thread group.) So, if you leave the thread group unspecified when you create your thread, the Java runtime system creates a ThreadGroup named main. Unless specified otherwise, all new threads that you create become members of the main thread group.

Our own Thread Group

Here we can also create our own thread group in which we can add our threads as and when we require. For creating a new thread we have ThreadGroup class and the object of this class will work as a Thread Group. Now at the time of creating the thread we have to specify this group name in the constructor of the Thread.

ex\ex79.java

Our own Thread Group

A thread group represents a set of threads. In addition, a thread group can also include other thread groups. The thread groups form a tree in which every thread group except the initial thread group has a parent.

Remember that : A thread is allowed to access information about its own thread group, but not to access information about its thread group's parent thread group or any other thread groups.

Various Methods of ThreadGroup

Method	Description
int activeCount()	Returns the no. of active threads in a group
int activeGroupCount()	Returns the no. of active groups in a group
void distroy()	Destroys this thread group and all of its subgroups.
String getName()	Returns the name of this thread group.
ThreadGroup getParent()	Returns the parent of this thread group.
boolean isDaemon()	Tests if this thread group is a daemon thread group.
boolean isDestroyed()	Tests if this thread group has been destroyed.
void list()	Prints information about this thread group to the standard output.
void setMaxPriority(int p)	Sets the maximum priority of the group.

Common Methods we can operate on ThreadGroup

The ThreadGroup class has three methods that allow you to modify the current state of all the threads within that group:

(1) resume
(2) stop
(3) suspend

These methods apply the appropriate state change to every thread in the thread group and its subgroups.

Daemon Thread

In Java, any thread can be a Daemon thread. Daemon threads are like a service providers for other threads or objects running in the same process as the daemon thread. Daemon threads are used for background supporting tasks and are only needed while normal threads are executing. If normal threads are not running and remaining threads are daemon threads then the interpreter exits.

Daemon Thread

Daemons are used for background tasks that make sense only when there are other threads that are doing the real work. The garbage collector is an excellent example of a proper use of daemons. You can set the daemon flag on or off as your program requires with thread.setDaemon(), **but you can only do this before you call start(). You cannot change the status of a running thread.** You can also check the status of a thread with isDaemon(). You will probably never use daemons.

Daemon Thread

There are two methods of thread for daemon

(1) void setDaemon (Boolean daemon)

(2) boolean isDaemon()

ex\ex80.java

Thank You